Object Oriented Programming (OOP)

[1] What are the core concepts of Object Oriented Programming?

Ans:

- The core concepts of Object Oriented Programming (OOP) include:

1. Encapsulation - Encapsulation is the mechanism of wrapping the data (variables) and code (methods) together as a single unit.

2. Inheritance - Inheritance is the process by which one class acquires the properties and behaviors of another class.

3. Polymorphism - Polymorphism allows one interface to be used for a general class of actions.

4. Abstraction - Abstraction is the concept of hiding the implementation details and showing only the essential features of the object.

---

5. Polymorphism - Allows one interface to be used for a general class of actions.

4. Abstraction - Abstraction is the concept of hiding the implementation details and showing only the essential features of the object.

@Mastercode.sagar

Features of Object Oriented Programming

1. **Encapsulation** - Encapsulation is the mechanism of binding data and methods together within a single unit (class).

2. **Inheritance** - Inheritance allows one class to inherit the fields and methods of another class.

3. **Polymorphism** - Polymorphism allows objects to be treated as instances of their parent class, enabling a single interface to represent different underlying forms (data types).

4. **Abstraction** - Abstraction refers to the concept of hiding the implementation details and showing only the essential features of an object.

5. **Reusability** - Reusability allows existing code to be used as a base for new programs, saving time and effort.

6. **Security** - Security is enhanced through encapsulation, which restricts access to the data within an object and protects it from unauthorized modification.

7. **Flexibility** - Flexibility is provided by polymorphism and inheritance, allowing programs to adapt and extend as needed.

8. **Maintainability** - Maintainability makes code easier to maintain and modify by using a modular structure that organizes data and functions into classes.

@Mastercode.sagar

Java OOP Example

Below is a Java example demonstrating object oriented programming concepts like encapsulation, inheritance, and polymorphism:

```
public class Animal {
    private String name;
    public Animal(String name) {
        this.name = name;
    }
    public String getName) {
        return name;
    }
}
public class Dog extends Animal {
    public Dog(String name) {
        Super (name);
    }
    public void bark() {
        System.out.println("name + " is barking");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy");
        dog.bark();
        System.out.println("Dog name: " + dog.getName());
    }
}
```

Output:

- Buddy is barking
- Dog name: Buddy

@Mastercode.sagar

Constructors in Java

- A constructor in Java is a special type of method that is automatically called when an object is created. Constructors are used to initialize the state of an object.

Types of Constructors

1. Default Constructor
   - A constructor with no parameters.
   - It initializes the object with default values.

2. Parameterized Constructor
   - A constructor that accepts arguments to initialize the object.
   - It allows the setting of initial values for object attributes.

3. No-Argument Constructor
   - A constructor that takes no arguments but is explicity defined by the programmer.
   - Similar to the default constructor, it initializes the object with default values.

4. Copy Constructor
   - A constructor that creates a new object as a copy of an existing object.
   - It is used to duplicate an object by copying the values of its attributes from another object.

Output:

@Mastercode.sagar

- Buddy is barking
- Dog name: Buddy

Method Overloading and Overriding in Java

1 Method Overloading (Compile Time Polymorphism)

- Method overloading allows multiple methods in the same class to have the same name but different parameters (different number, types or order).
- It is resolved at compile time.

Syntax:

```
returnType  methodName(parameter1, parameter2) {
        // method body
}
```

Example:

```
public class Calculator {
    public int add(int a, int b){  return a + b; }
    public int add(int a, int b, int c){  return a + b + c; }
    public double add(double a, double b){  return a + b; }
}
```

2 Method Overriding (Run Time Polymorphism)

- Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass.
- The method name, return type and parameters must be the same as in the parent class.
- It is resolved at run time (dynamic method dispatch).

Syntax:

```
class Subclass extends Superclass {
    @Override
        returnType  methodName(parameters) {
            // method body
        }
}
```

Flow:

↓

Superclass ⟷ Subclass

↓

✓ Method is overridden in subclass.

Example:

```
class Animal {
    void sound() {  System.out.println("Animal makes sound"); }
}
class Dog extends Animal {
    @Override
        void sound() {  System.out.println("Dog barks"); }
}
```

Key Difference:

@Mastercode.sagar

Overloading → Same class, same method name, different parameters.	**Overriding** → Different class, same method signature, specific implementation.

Access Modifiers in Java

- Access modifiers in Java determine the scope of variables, methods, and classes. They control the visibility of these members to other parts of the program.

Access Modifiers in Java

Modifier	Scope	+ Subclass	+ World
Private	Within the same class	✗	✗
Default	Within the same package	✗	✗
Protected	Same package + subclasses (even in different packages)	✓	✗
Public	From everywhere	✓	✓

✓ = Accessible
✗ = Not Accessible

2. Method Overriding :-

Modifier	Scope	+ Subclass	+ World
Private	Within the same class	✗	✗
Default	Within the same package	✗	✗
Protected	Same package + subclasses (even in different packages).	✓	✗
Public	From everywhere	✓	✓

@Mastercode.sagar

Access Modifiers in Java

- Access modifiers in Java determine the scope of variables, methods, and classes. They control the visibility of these members to other parts of the program.

Access Modifiers in Java

Modifier	Scope	+ Subclass	+ World
Private	Within the same class	✗	✗
Default	Within the same package	✗	✗
Protected	Same package + subclasses (even in different packages)	✓	✗
Public	From everywhere	✓	✓

✓ = Accessible
✗ = Not Accessible

1. Encapsulation :-

- Encapsulation is one of the fundamental concepts in object oriented programming.
- It is the technique of wrapping the data (variables) and code (methods) together into a single unit.
- Encapsulation helps to hide the internal state of an object from the outside.
- The internal data of an object can be accessed only through the public methods of the class.

@Mastercode.sagar

Inheritance in Java

- Inheritance is a mechanism in OOP that allows a class (subclass.) to inherit the properties (fields and methods) of another class (superclass.).
- It represents the IS-A relationship.
- It helps in code reusability and method overriding.

Syntax :

```
class Subclass extends Superclass { }
```

Key Points :

» The subclass inherits all public and protected members of the superclass.

» Private members are not inherited.

» A class can inherit from only one class (single inheritance in Java).

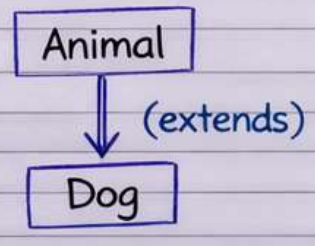
» The subclass can add naw fields and methods or override existing ones.

Types of Inheritance :

1. **Single Inheritance** – One subclass inherits from one superclass.
2. **Multilevel Inheritance** – A class inherits from a class, which in turn inherits from another.
3. **Hierarchical Inheritance** – Multiple subclasses inherit from one superclass.
4. **Hybrid Inheritance** – Combination of two or more types (not supported directly in Java using classes, but possible through

Example :

```
class Animal {                          ← Superclass.
    void eat(.) { ... }
}
class Dog extends Animal {              ← Subclass
    void bark(.) { ... }
}
```



@Mastercode.sagar

Polymorphism in Java

- Polymorphism is the ability of an object to take many forms, It allows methods to do different things based on the object that is calling them.

Types of Polymorphism :-

1. Compile-Time Polymorphism (Static Binding)

> Also known as method overloading.

> It occurs when multiple methods have the same name but differ in the number or type of parameters.

> It helps in code reusability and method overriding.

Syntax :

```
class Calculator extends Superclass { }
```

Key Points :

Method Overloading →

```
class Calculator
  int add(int a, int b) {..}
  double add(double a,
        double b) ( ...) {..}
}
```

> The subclass inherits all public and protected members of the superclass.

> Private members ore not inherited.

> A class can inherit from only one class (single inheritance in Java).

> The subclass can add new fields and methods or override existing ones.

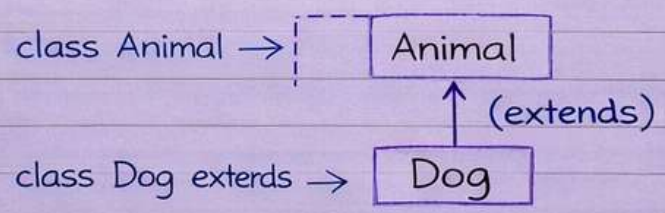
Polymorphism Advantages :-

> Improves code reusability and readability.

> Allows for a single interfoce to represent multiple types.

> Makes it easier to maintain and extend code.

> Supports method overriding and method overloading.



Run-Time Polymorphism

- Run-time polymorphism in Java is achieved through method overriding and is based on dynamic method dispatch.

Method Overriding :-

Key Points :

> Method overriding occurs in inheritance, when a subclass provides a specific implementation of a method that is already defined in its superclass.

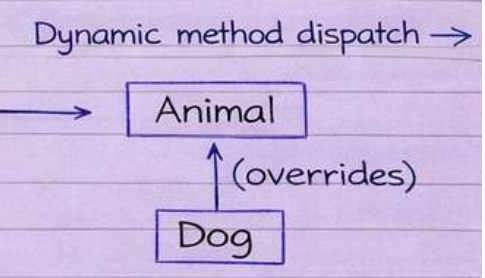
> The method in the subclass must have the same name, return type, and parameters as the method in the superclass.

> It helps in achieving runtime polymorphism, allowing a method to be used based on the actual object type that is instantiated.

> A reference variable of a superclass can refer to an object of a subclass and invoke the overridden method.

```
class Animal {
  void makeSound() { ... }
    System.out.println("Animal makes a sound");
}
```

```
class Dog extends Animal {
  @Override  void makeSound() { ... }
    System.out.println("Dog barks");
}
```



Output :

- Dog barks

- In this example, the reference variable "myDog" of type Animal is assigned to a "Dog" object. When "myDog.makeSound()" is called, it executes the overridden makeSound() method in the "Dog' subclass, outputting 'Dog barks'. This is an example of dynamic method dispateh.

@Mastercode.sagar

Abstract Classes & Interfaces in Java

- Run-time polymorphism in Java is achieved through method overriding and is based on dynamic method dispatch.

1) Abstract Classes :-

> An abstract class cannot be instantiated, i.e. you cannot create objects of an abstract class.

> It can have abstract methods (without implementations) as well as concrete methods (with implementations).

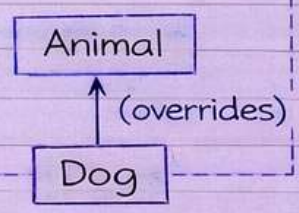
> Abstract methods must be implemented in a subclass.

> It can have constructors, fields, and methods like any other class.

> It is used to provide a common base for derived classes.

Syntax :

```
abstract class Animal {                          <— abstract method
    abstract void makeSound(); // astract method
    void eat() {
        // concrete method implementation
}
```



2) Interfaces :-

> An interface cannot be instantiated, i.e. you cannot create objects of an interface.

> It can only have abstract methods (implicitly public).

> Fields in interfaces are by default static, final and and public.

> A class implements an interface using the "implements" keyword and must provide implementations for all its abstract methods.

Syntax :

```
interface Animal {
    void makeSound() {   // abstract method (implicitly public)
    class Dog implements Animal {
        public void makeSound() { .. }
}
```

Packages & Import in Java

1) Package in Java :-

- A package is a collection of related classes and interfaces.
- It helps in organizing code and avoiding name conflicts.
- Packages provide access protection and improved code reusability

Syntax :

```
package packageName;
```

Built-in Packages in Java :-

- java.lang (default package) | - java.io (Input/Output)
- java.util (Utilities, Scanner, etc.) | - java.net (Networking)

2) Import in Java :-

- The import keyword is used to use classes from other packages
- It removes the need to write the full package name.

Syntax :

```
import packageName.ClassName;
```
→ To import a specific class.

```
import packageName.*;
```
→ To import all classes in the package

Example :

```
package demo;
import java.util.Scanner;
public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
    }
```

← Using Scanner from java.util

Advantages of Packages :-

1. Avoids naming conflicts
2. Easy access control
3. Code reusability and organization.
4. Enhances security (with access modifiers)

@Mastercode.sagar